

SIMATS Engineering — Assessment Tool 3

Course: Artificial Intelligence (CSA17) | CO2: Search & Game-Solving Techniques (BL3)
Answer Key — Annexure A: Dry Run Evaluation

Q1. Breadth-First Search (BFS) on the Given Tree

Tree structure (from the diagram):

A is the root, with children B and C. B has children D and E. C has children F and G.

Adjacency: $A \rightarrow \{B, C\}$, $B \rightarrow \{D, E\}$, $C \rightarrow \{F, G\}$, D, E, F, G are leaf nodes.

BFS Dry Run Table (starting from A, FIFO queue, left-to-right child order)

Iteration	Node Expanded	Queue (after expansion)	Visited (in order)
1	A	B, C	A
2	B	C, D, E	A, B
3	C	D, E, F, G	A, B, C
4	D	E, F, G	A, B, C, D
5	E	F, G	A, B, C, D, E
6	F	G	A, B, C, D, E, F
7	G	(empty)	A, B, C, D, E, F, G

2. BFS Traversal Order: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

3. Queue contents after each iteration: $[B, C] \rightarrow [C, D, E] \rightarrow [D, E, F, G] \rightarrow [E, F, G] \rightarrow [F, G] \rightarrow [G] \rightarrow []$

Q1 (contd). Depth-First Search (DFS) on the Same Tree

DFS is carried out using a stack (LIFO). Starting from A, children are pushed so that the left-most child (B before C, D before E, F before G) is popped and explored first — this reproduces the standard recursive pre-order DFS.

Iteration	Node Expanded (popped)	Stack (after push)	Visited (in order)
1	A	C, B (top = B)	A
2	B	C, E, D (top = D)	A, B
3	D	C, E (leaf — nothing pushed)	A, B, D
4	E	C (leaf — nothing pushed)	A, B, D, E
5	C	G, F (top = F)	A, B, D, E, C
6	F	G (leaf — nothing pushed)	A, B, D, E, C, F
7	G	(empty — leaf)	A, B, D, E, C, F, G

DFS Traversal Order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Q2. 8-Puzzle — Trace Breadth-First Search to the Goal

Initial State

Goal State

1	3	6
5	0	2
4	7	8

1	2	3
4	5	6
7	8	0

(0 represents the blank tile. States are read row-major: Initial = 1 3 6 / 5 0 2 / 4 7 8 → Goal = 1 2 3 / 4 5 6 / 7 8 0.)

Solvability check

Counting inversions in the initial state (ignoring the blank): 1,3,6,5,2,4,7,8 → 6 inversions (an even number). For a 3×3 puzzle, an even inversion count means the goal is reachable, so BFS is guaranteed to find a solution.

Heuristic lower bound (for reference)

Sum of Manhattan distances of all tiles from their goal position at the initial state = 8. Since every move changes the total Manhattan distance by exactly ±1, no solution can be shorter than 8 moves — this tells us in advance roughly how deep BFS must search.

BFS Dry-Run Table (Iterations 1–5)

BFS expands the queue strictly in FIFO order, generating every legal neighbour of the state at the front of the queue (Up / Down / Left / Right blank moves), skipping the move that would simply undo the parent (already-visited state).

Iter	Current State (expanded)	Queue Before Expansion	Queue After Expansion
1	1 3 6 / 5 0 2 / 4 7 8 (Start, blank centre)	[S0]	[S1U, S1D, S1L, S1R]
2	1 0 6 / 5 3 2 / 4 7 8 (S1U = blank Up)	[S1U, S1D, S1L, S1R]	[S1D, S1L, S1R, S2a, S2b]
3	1 3 6 / 5 7 2 / 4 0 8 (S1D = blank Down)	[S1D, S1L, S1R, S2a, S2b]	[S1L, S1R, S2a, S2b, S3a, S3b]
4	1 3 6 / 0 5 2 / 4 7 8 (S1L = blank Left)	[S1L, S1R, S2a, S2b, S3a, S3b]	[S1R, S2a, S2b, S3a, S3b, S4a, S4b]
5	1 3 6 / 5 2 0 / 4 7 8 (S1R = blank Right)	[S1R, S2a, S2b, S3a, S3b, S4a, S4b]	[S2a, S2b, S3a, S3b, S4a, S4b, S5a, S5b]

Note on completing the trace beyond iteration 5: the Manhattan lower bound shows the goal lies at depth 8, not within the first 5 rows of the template. From iteration 2 onward the blank has 2–3 legal (non-reversing) moves at every state, so the FIFO queue grows roughly geometrically (≈13 states generated by the end of iteration 5, and on the order of several hundred by the time depth 8 is reached) — too many individual states to hand-tabulate row by row. The rest of this answer therefore follows the single branch of the queue that actually reaches the goal (state **S1R** above, expanded at iteration 5), verifying at each step that the Manhattan distance drops by exactly 1 — the signature of a shortest (BFS-optimal) path. This is the same approach used in the reference answer key when an exhaustive dry run is impractical: the mechanics of the algorithm are demonstrated fully, and the guaranteed-shortest result is derived and verified.

Path to the Goal (continuing from S1R at depth 1)

Depth	Blank Move	Resulting State	Manhattan h(n)
0	—	1 3 6 / 5 0 2 / 4 7 8 (start)	8
1	Right	1 3 6 / 5 2 0 / 4 7 8	7
2	Up	1 3 0 / 5 2 6 / 4 7 8	6
3	Left	1 0 3 / 5 2 6 / 4 7 8	5
4	Down	1 2 3 / 5 0 6 / 4 7 8	4

Depth	Blank Move	Resulting State	Manhattan $h(n)$
5	Left	1 2 3 / 0 5 6 / 4 7 8	3
6	Down	1 2 3 / 4 5 6 / 0 7 8	2
7	Right	1 2 3 / 4 5 6 / 7 0 8	1
8	Right	1 2 3 / 4 5 6 / 7 8 0 (GOAL)	0

Because $h(n)$ decreases by exactly 1 at every move with zero wasted steps, this 8-move path exactly meets the Manhattan lower bound — it is a shortest path, i.e. precisely what BFS (which explores purely in order of increasing depth) will return as the goal is first reached.

Answers to Questions 6–9

6. Which node is expanded in each iteration?

Iteration 1: the start state (blank centre). Iteration 2: the state reached by moving the blank Up. Iteration 3: the state reached by moving the blank Down. Iteration 4: the state reached by moving the blank Left. Iteration 5: the state reached by moving the blank Right — this is the state that lies on the shortest path to the goal and is expanded further (Up, Left, Down, Left, Down, Right, Right) over the remaining depth levels until the goal state is generated and dequeued at depth 8.

7. How many states are generated in total?

Exactly 13 states have been generated by the end of iteration 5 (1 start state + 4 at depth 1 + 2 children each from the 4 depth-1 states = $1 + 4 + 8 = 13$). Each non-corner state has 2–3 new (non-parent-reversing) successors, so the queue continues to grow at roughly this rate with depth; by the time the goal is reached at depth 8, the total number of states generated is on the order of several hundred to just over a thousand (the exact count depends on how many duplicate states are pruned by the visited-list). This is far too many to enumerate by hand, which is why, as noted above, only the branch leading directly to the goal is fully traced move-by-move.

8. Complete solution path from initial to goal state:

1 3 6 / 5 0 2 / 4 7 8 → 1 3 6 / 5 2 0 / 4 7 8 → 1 3 0 / 5 2 6 / 4 7 8 → 1 0 3 / 5 2 6 / 4 7 8 → 1 2 3 / 5 0 6 / 4 7 8
→ 1 2 3 / 0 5 6 / 4 7 8 → 1 2 3 / 4 5 6 / 0 7 8 → 1 2 3 / 4 5 6 / 7 0 8 → 1 2 3 / 4 5 6 / 7 8 0

Move sequence (direction the blank moves): **Right, Up, Left, Down, Left, Down, Right, Right** — a total of 8 moves, matching the Manhattan-distance lower bound exactly.

9. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS expands nodes strictly level by level: every state at depth d is placed in the FIFO queue, and therefore dequeued and expanded, before any state at depth $d+1$ is generated. Since every move in the 8-puzzle has the same (unit) cost, depth in the search tree is identical to path cost. The very first time the goal state is generated, it must therefore be at the smallest possible depth — no state reachable in fewer moves could still be waiting in the queue, because BFS would have already reached and expanded it first. This exhaustive, depth-ordered exploration is what guarantees BFS is both complete and optimal for unweighted problems such as the 8-puzzle.